

基于NVIDIA Hopper 架构的LLM长文 本推理场景性能优化实践和探索

Jeson Xiang

向乾彪

2026.03.17

大纲目录

腾讯混元及长文场景介绍

稀疏注意力机制探索

长文并行策略优化

Attention Kernel 计算优化

长文 KVCache 优化

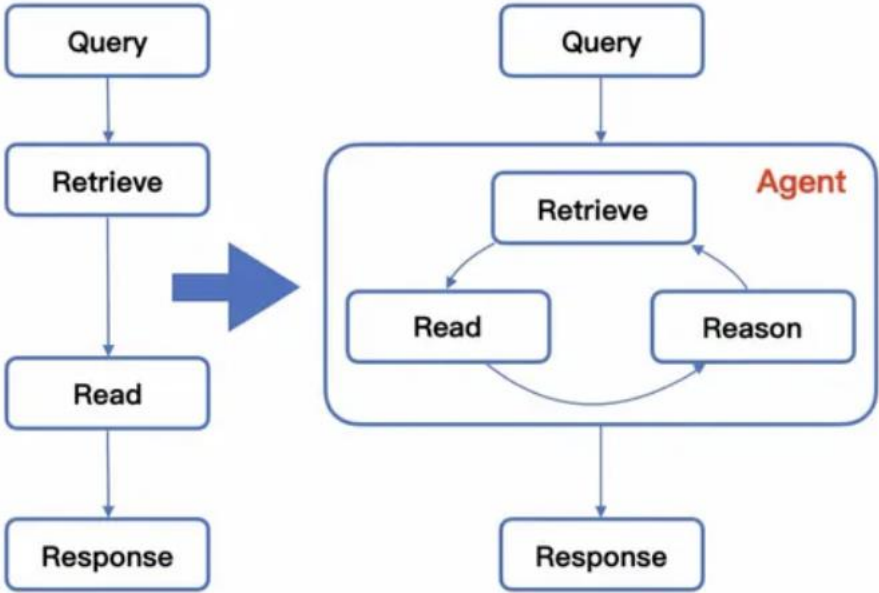
总结

腾讯长期持续投入混元大模型研发

做技术的长期主义者
兼顾实用和创新
坚持全链路自研



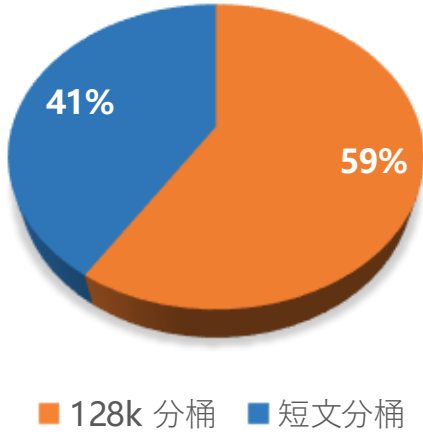
混元长文推理成本居高不下



一、元宝 Deepsearch 模型，planner 模型触发多轮搜索后 token 长度达到 30k，下游 summary 模型作为主模型前置链路，首字耗时对整个链路的时耗非常关键

二、元宝主模型场景，128k 长文分桶消耗的 GPU 计算资源占比高达 60%，成本压力非常大，需要做极致的性能优化来降低线上成本。

线上资源占比



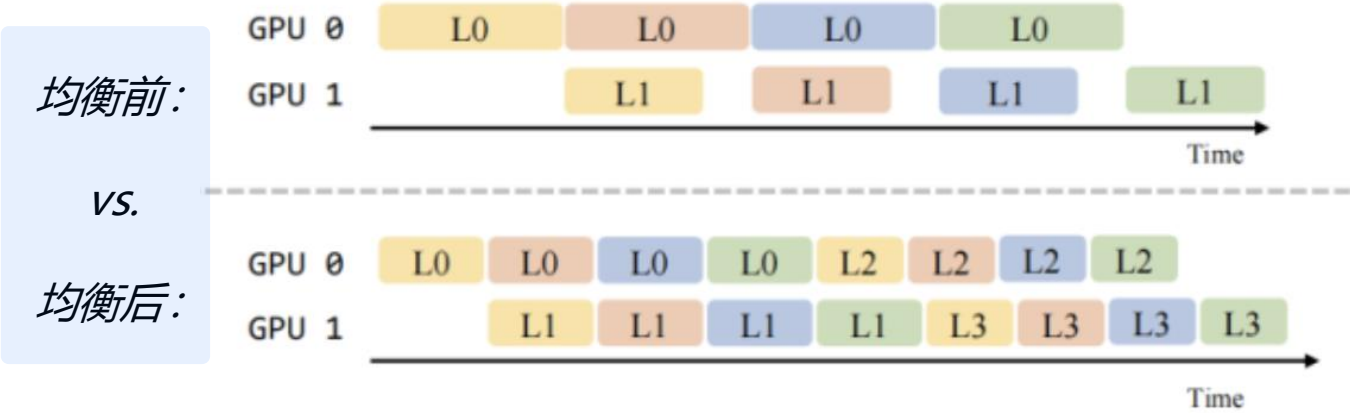
LLM长文推理几大痛点

➤ Attention 计算为平方复杂度

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

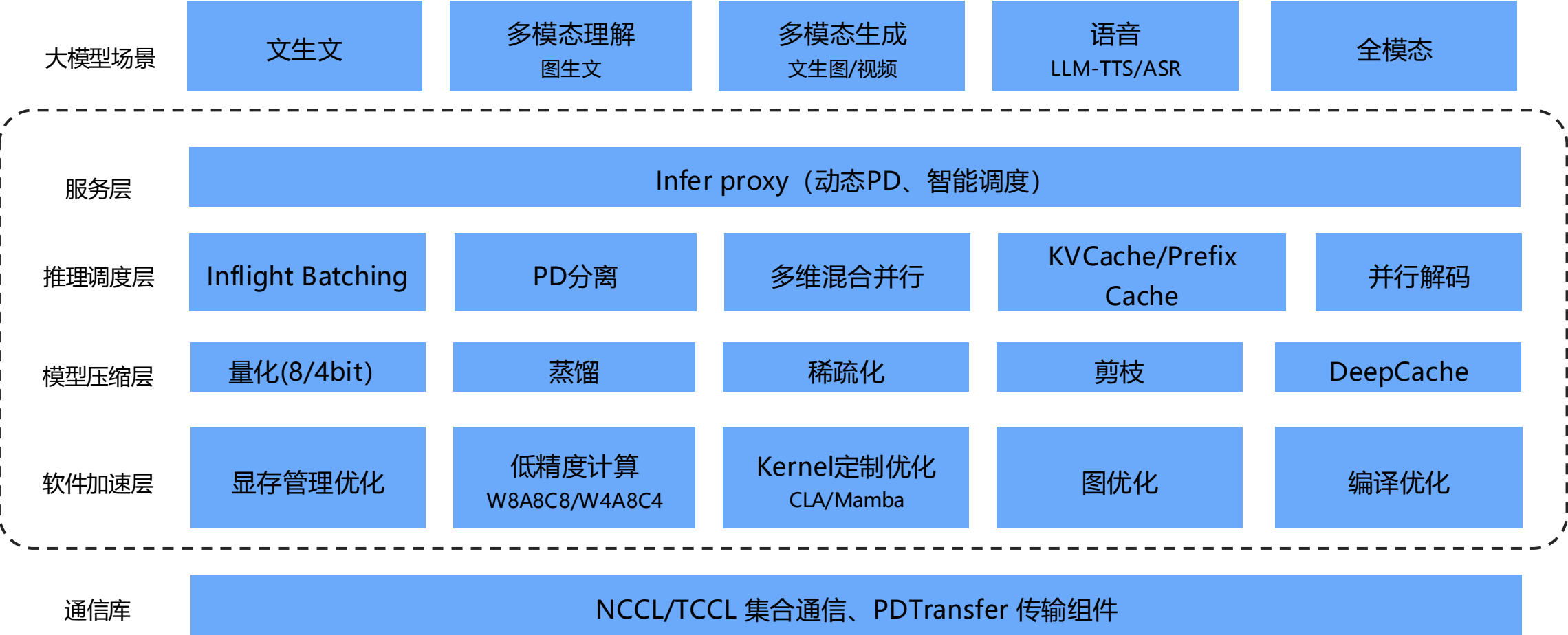
➤ 常见的多机并行策略，超长文场景负载
均衡问题

➤ KVCache 存储量大，访存及通信成本高



AngelHCF大模型推理框架

- AngelHCF 框架：全面支持腾讯混元大模型的研发及业务落地部署
- 全链路定制优化，相比业界开源框架性能提升1.3倍



大纲目录

腾讯混元及长文场景介绍

稀疏注意力机制探索

长文并行策略优化

Attention Kernel 计算优化

长文 KVCache 优化

总结

稀疏注意力机制 – 文生文

背景:

- 长上下文(32k及以上)的prefill计算量随序列长度大幅增长
- Attention稀疏能够大幅度减少计算量

挑战:

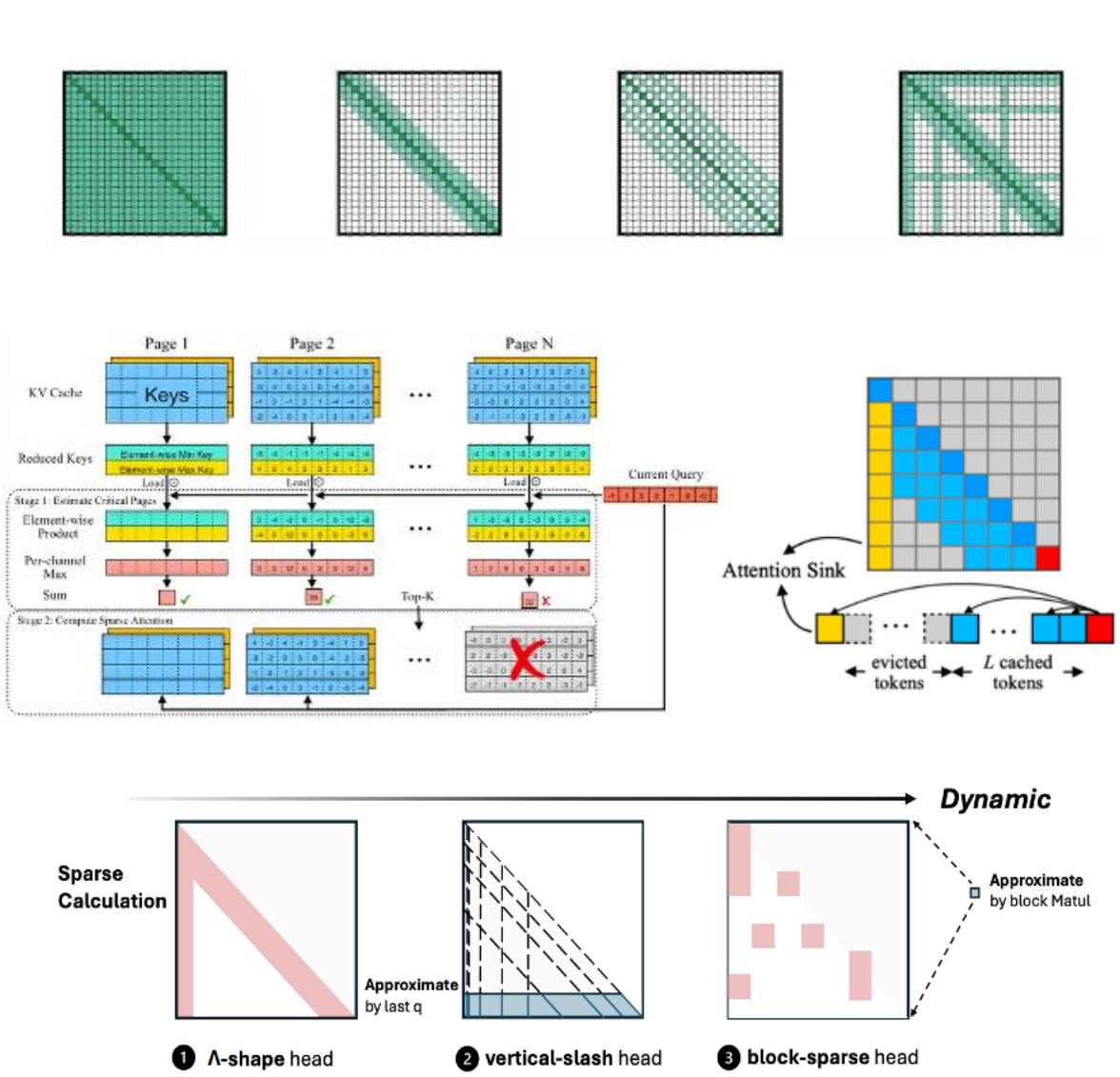
- 不同Attention块稀疏设计不同
- 稀疏策略需要数据支撑

解决:

- 稀疏模式搜索算法: 为每个Attention自动化的搜索出最佳的稀疏模式及稀疏超参
- Training-free: 使用预定义的结构约束
- Trainable: NSA及MoBA注意力分块策略

收益: 长文性能加速25%

BS	Speed Up
2	1.15x
4	1.15x
8	1.08x
12	1.25x



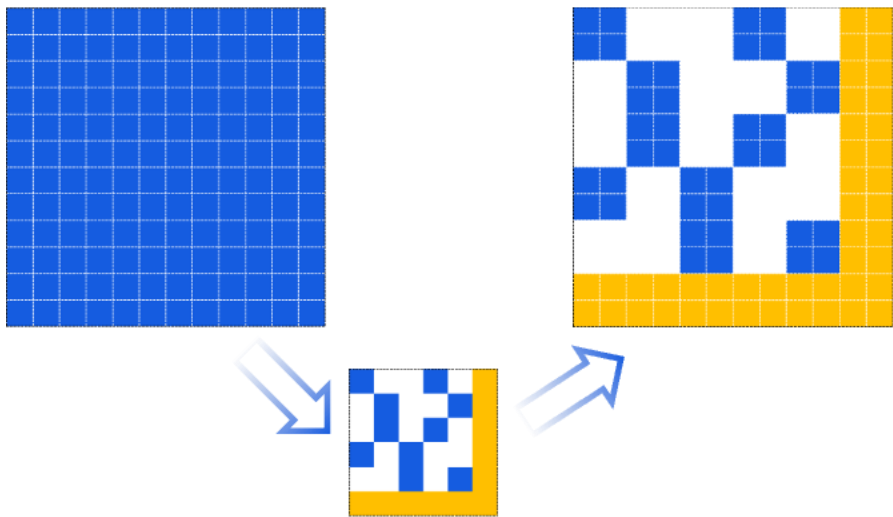
稀疏注意力机制 – 多模态

问题与挑战:

- 视频生成面临输入序列超长问题
- 扩散模型compute-bound
- 注意力稀疏: 70%注意力分数仅来自于15%的token空间

业界做法:

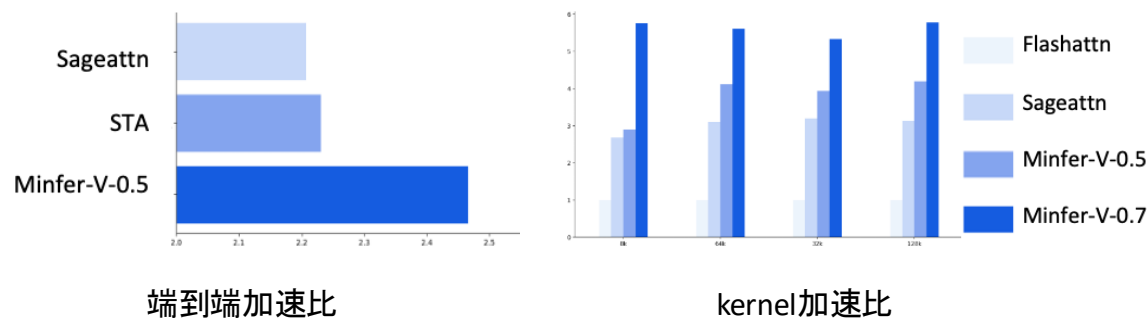
- 灵活性差: 序列长度须为预定选项或某数倍数
- 易用性差: 硬件兼容性差
- 与Cache跳步不兼容



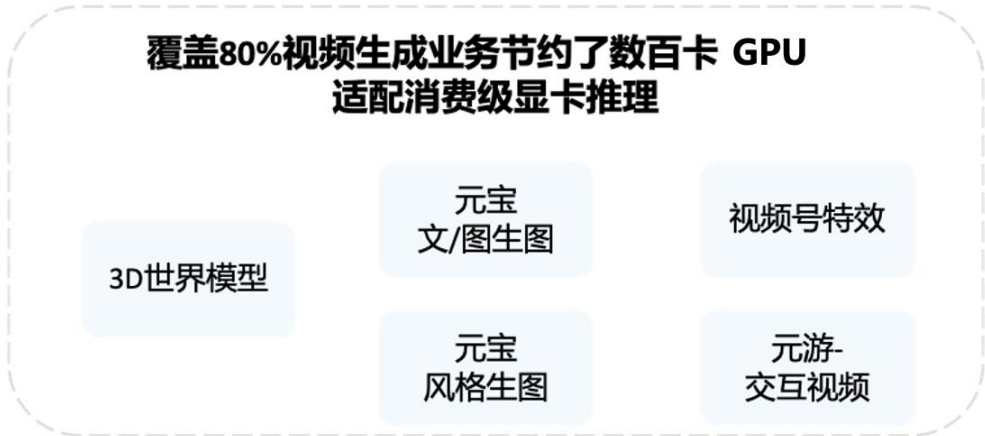
优化算法:

- 应用Text sink: 不对文本信息对应的序列进行稀疏
- 基于Flash-Attention3自研, 结合Hopper深度优化
- 稀疏量化结合: 双管齐下优化耗时

加速收益:



业务落地:



稀疏注意力机制 – 算法探索创新

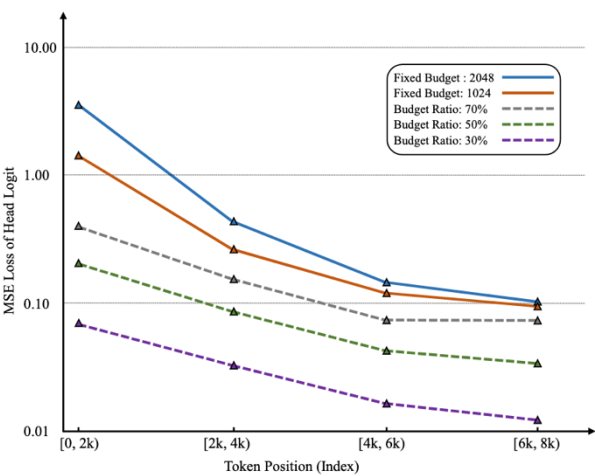
问题发现：

- 位置靠前的token更重要，中后位置的token低敏感

优化思路：

$$\mathcal{M}_{i,j} = \underbrace{Q_i K_j^T}_{\text{Routing}} + \underbrace{\beta \cdot \max(0, \log(\|V_j\|_2))}_{\text{Magnitude}}$$

- 根据P矩阵输出和log(v)来计算block的Metric，并利用topk筛选重要的block



效果：Qwen等模型在RULER/Longbench上效果SOTA

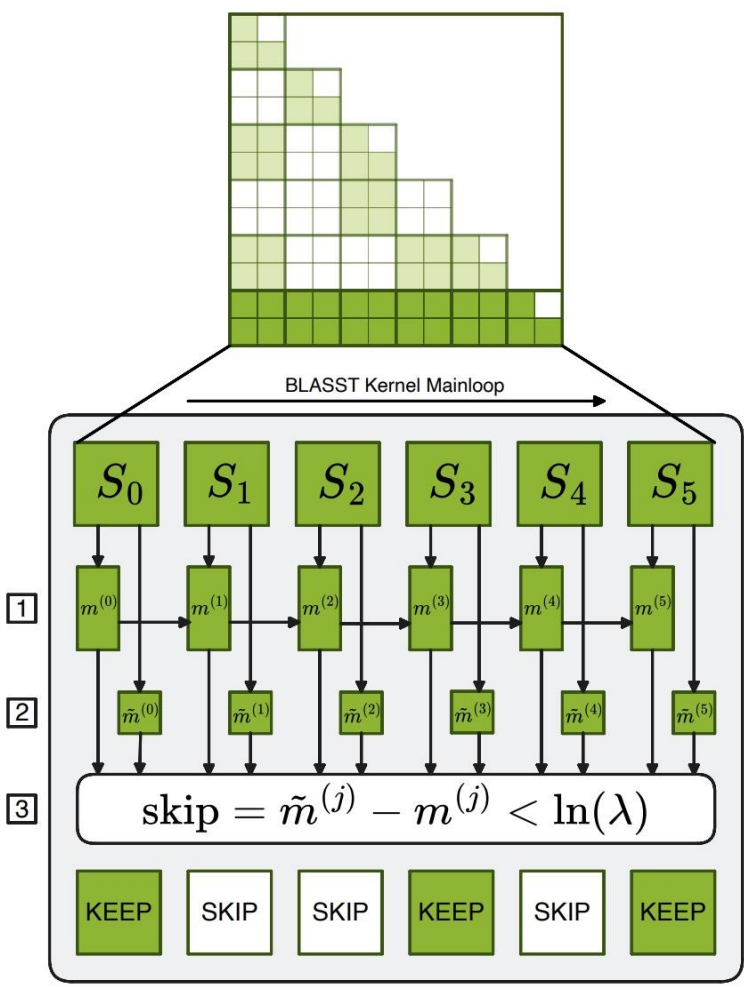
METHOD	4K	8K	16K	32K	64K	128K	AVG	BUD.
LLAMA-3.1-8B								
DENSE	95.52	94.06	93.43	88.19	84.86	77.11	88.86	100%
MINF	95.51	93.56	92.70	91.68	80.06	76.67	88.36	55%
FLEX	94.98	93.91	92.68	89.51	82.66	75.40	88.19	27%
XATTN	95.46	93.83	93.65	89.82	81.60	74.35	88.12	26%
STEM	95.17	93.96	93.73	89.92	82.67	75.37	88.47	25%
QWEN3-8B								
DENSE	95.44	93.08	92.71	92.12	79.57	73.03	87.66	100%
MINF	95.01	92.56	91.70	91.68	77.54	71.87	86.73	76%
FLEX	95.06	91.98	91.17	90.93	77.31	70.28	86.12	30%
XATTN	95.03	92.56	91.76	91.72	78.60	71.80	86.91	35%
STEM	95.18	92.68	92.11	91.85	78.78	72.31	87.15	25%

收益：DeepSeek v3.2上减少30%Attention Block并无损

Model	DS 3.2	DS 3.2+ Ours
Sparsity Ratio	N*2048	(N*2048)*0.7
Metric	Score	Output
Ceval Long_qa	55.6	56.7
Ceval Long_cm	83.2	81.8
Ceval Long_ss	18.4	18.4
Ceval New_label	88.9	87.1
Ceval Key_passage	98.1	98.3
Ceval Table_query	90.6	89.7

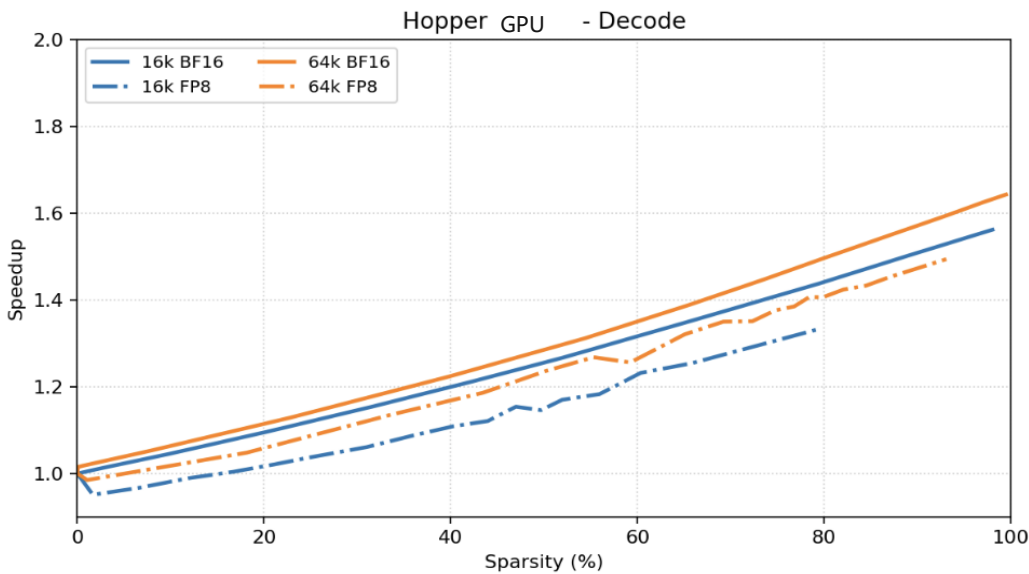
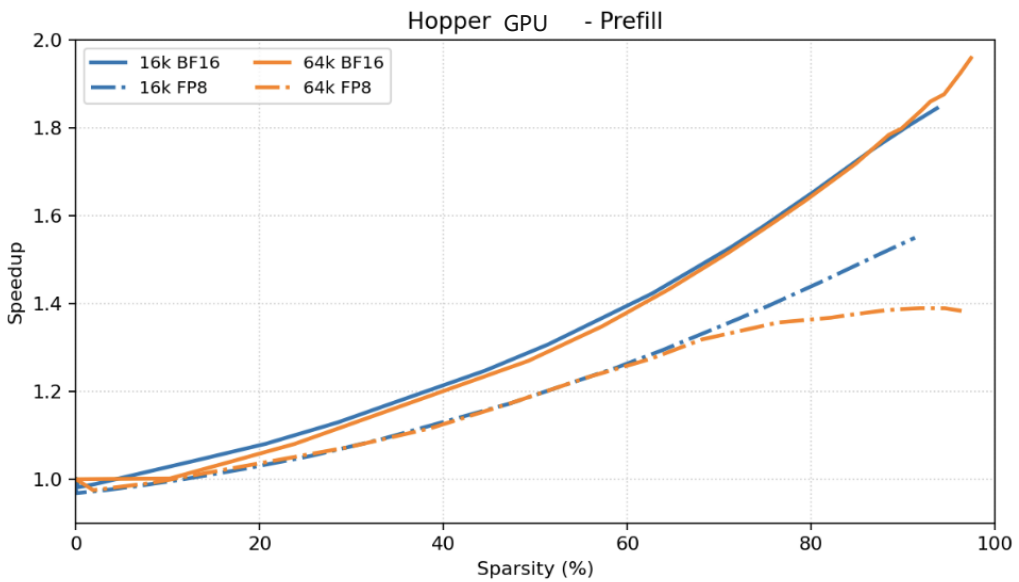
稀疏注意力机制 – Skip-Softmax FMHA

➤ 针对长文稀疏化与 NV 的合作&探讨



Prefill&Decode
均有收益

与其他优化
兼容性好



大纲目录

腾讯混元及长文场景介绍

稀疏注意力机制探索

长文并行策略优化

Attention Kernel 计算优化

长文 KVCache 优化

总结

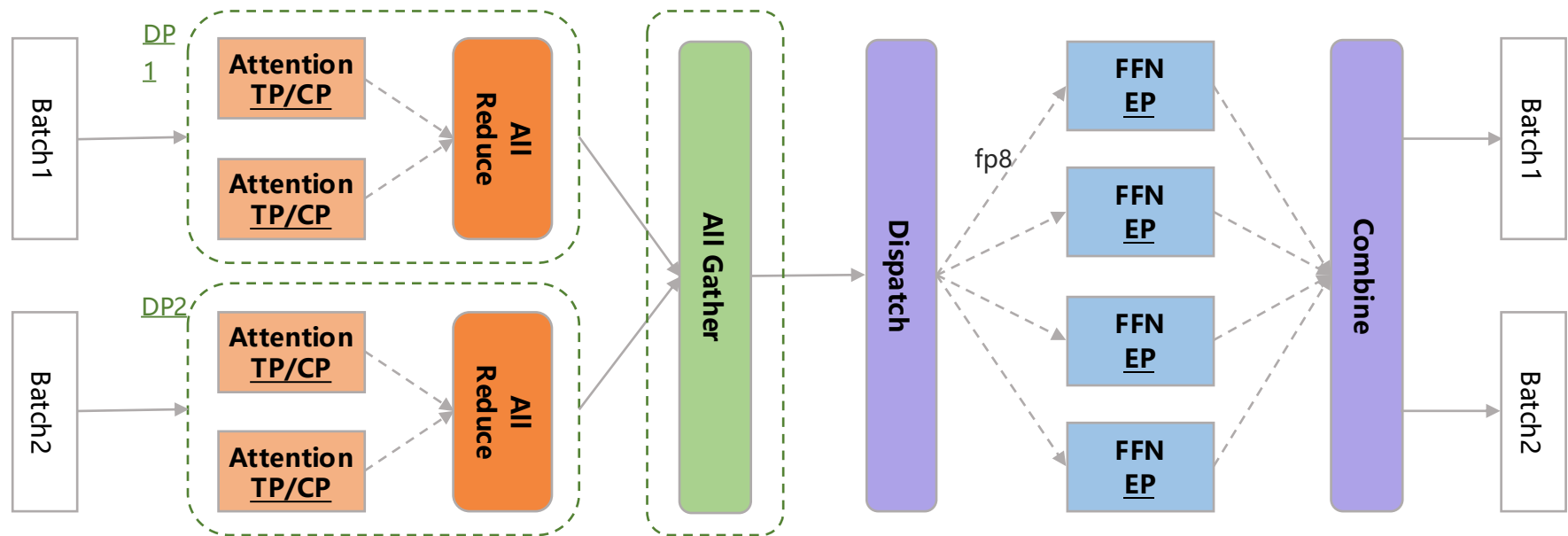
长文并行策略优化 – 多维并行策略

针对 Prefill

算力瓶颈，主要采用 大TP&CP + 小EP
降低首字时耗，减少跨机通信量

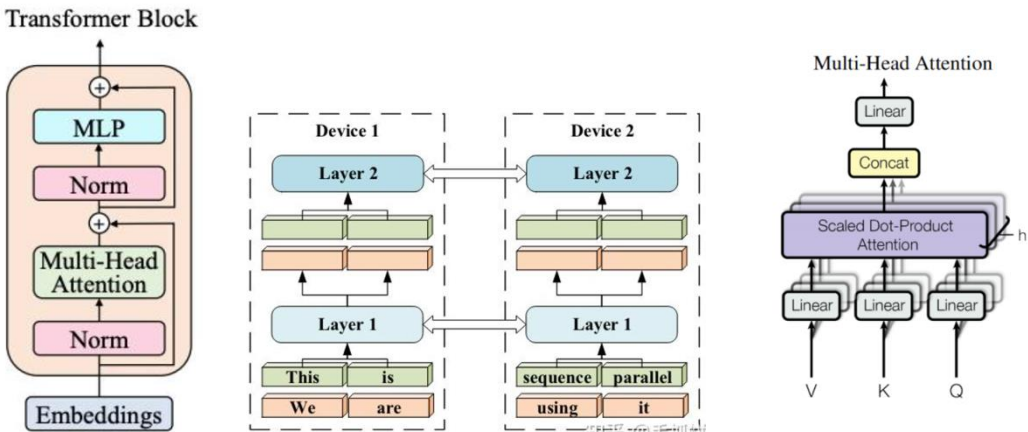
针对 Decode

访存瓶颈，主要采用 DP + 大EP，减少单卡访存压力
充分利用显卡算力，趋近 Compute bound



长文并行策略优化 – TP+CP并行

序列并行(CP): 在序列维度进行切分到多个计算设备并行计算



分布式Attention

- DeepSpeed-Ulysses

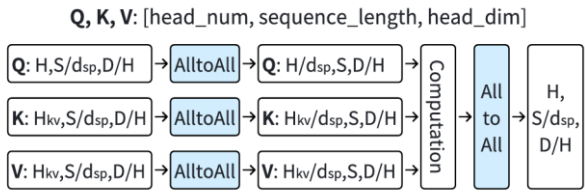
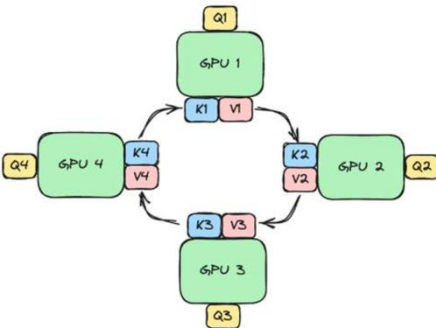


Figure 2. Ulysess-Attention performs head-parallel computation across GPUs with two steps of AlltoAll.

- Ring-Attention



分布式的FlashAttention

Token level类型算子: Linear, LayerNorm、MLP

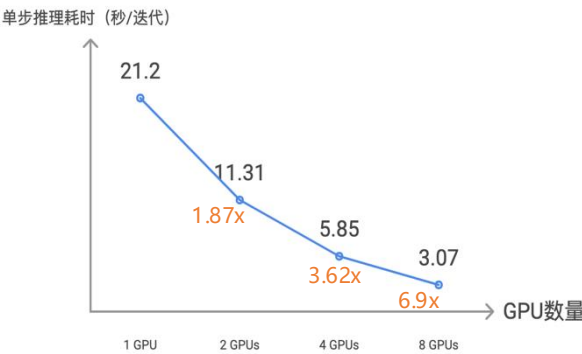
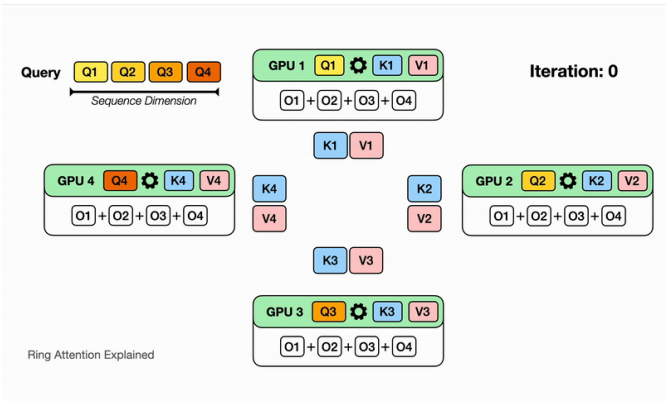
- token间无需交叉计算, 适合序列并行

Attention算子: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d}})V$,

- token间存在交叉计算
- Softmax之后的P矩阵计算依赖全量的query和value, P矩阵乘value矩阵也需要全量value矩阵

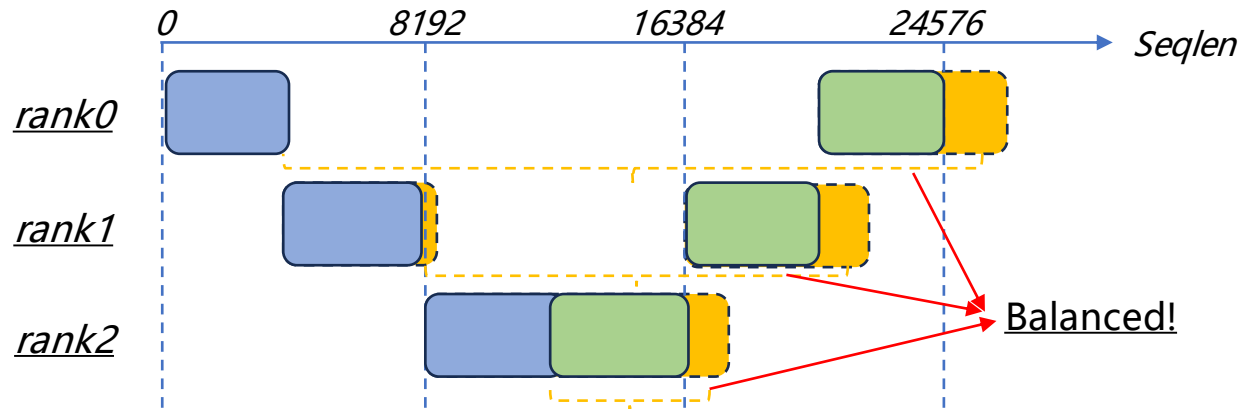
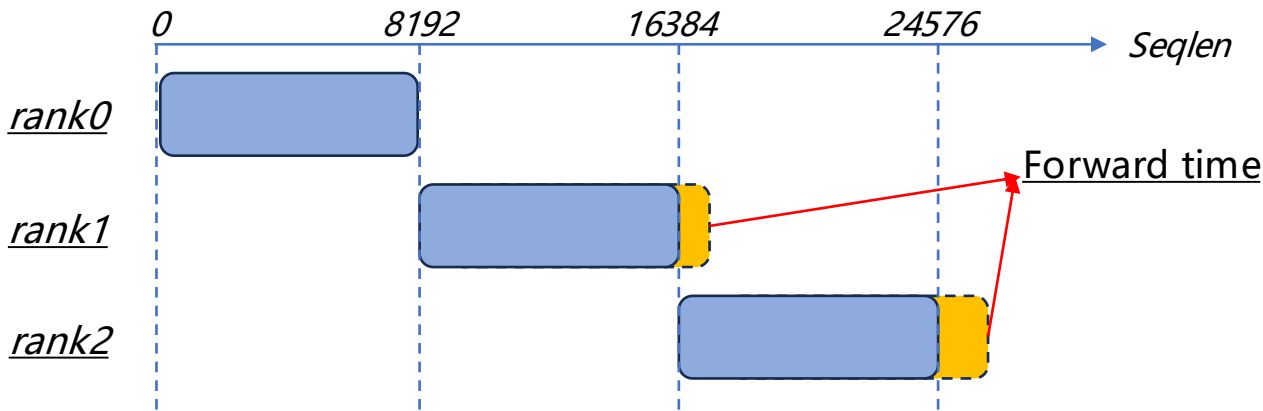
核心是Attention如何做序列并行, 即**分布式Attention**

Ring-Attention



卡数增加时, 加速收益变高, 趋向于线性

长文并行策略优化 – CP均衡优化



问题:

切 CP 后越靠后的 rank 分到的 position 越长，推理时耗更长，造成 rank 间等待

解法:

优化 CP 负载均衡问题，切分两倍分块，头尾分块结合均衡 rank 推理耗时

☆ 长文收益:

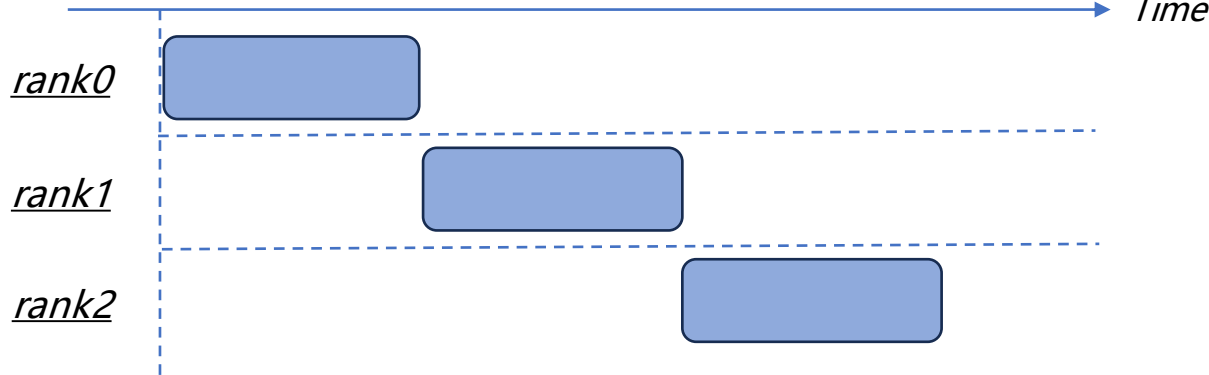
	TP	CP	CPLB	加速比
3.1k	0.323s	0.185s		1.74倍
9.3k	0.862s	0.364s		2.368倍
23k	2.260s	0.816s		2.76倍
58k	6.533s	2.143s	2.027s (-5.6%)	3.04倍
71k	8.725s	2.768s	2.626s (-5.2%)	3.16倍

长文并行策略优化 – 跨机CPP优化

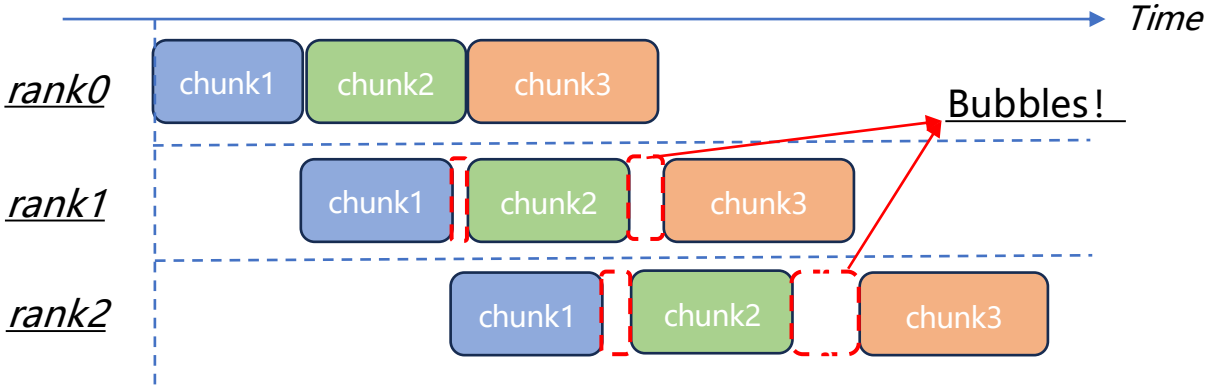
问题:

超长文需要通过多机推理来提升并行度，Pipeline Parallel 能有效降低跨机通信量，但需要增加 chunk 切分来优化 TTFT，chunk 间容易有bubble问题

Pipeline Parallel



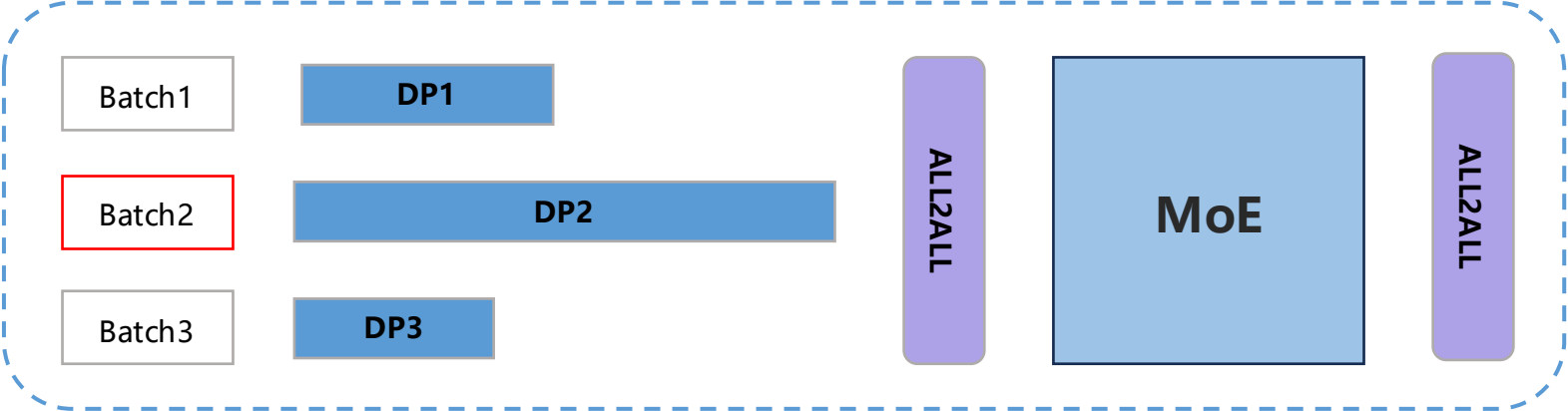
Chunked Pipeline Parallel



解法:

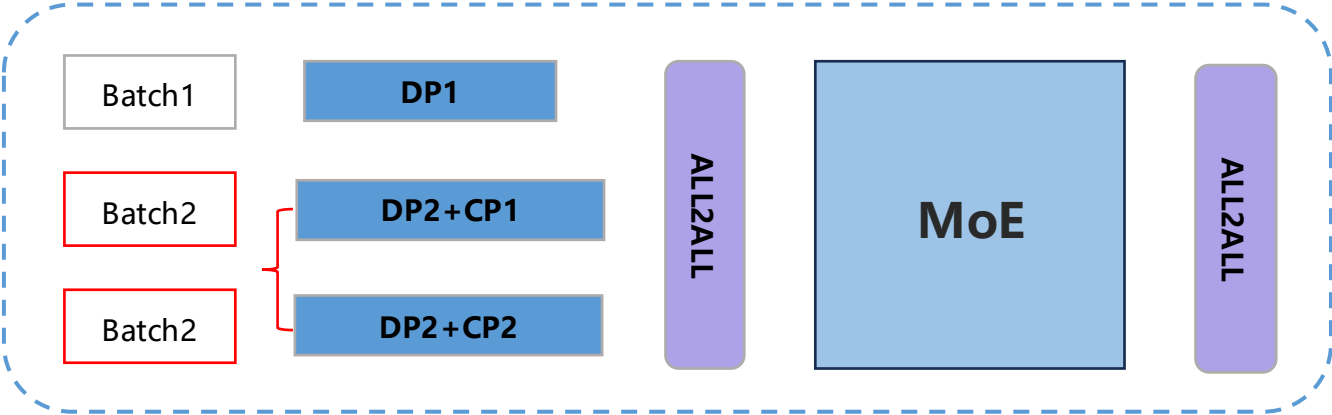
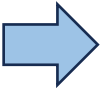
根据 Prefill 长度做动态 chunk 切分策略，减少不同请求 chunk 位置推理耗时的 Bubble 问题

长文并行策略优化 – DP+CP优化负载



问题:
Attention DP 后容易存在长文负载不均问题，单个 rank seqlen 过长会严重拖慢其他 DP rank

解法:
针对Attention DP 的负载不均问题，根据 seqlen 增加 CP 切分，均衡各 DPrank 处理时耗



大纲目录

腾讯混元及长文场景介绍

稀疏注意力机制探索

长文并行策略优化

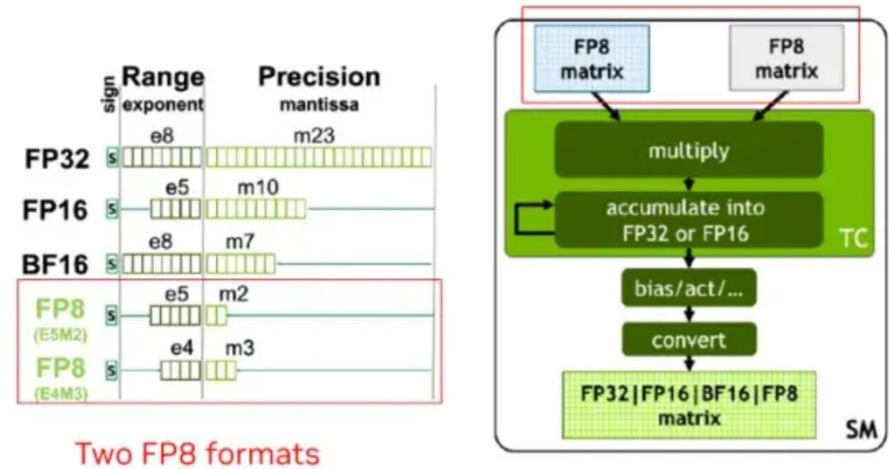
Attention Kernel 计算优化

长文 KVCache 优化

总结

长文Attention计算优化 – FP8量化

What is FP8?



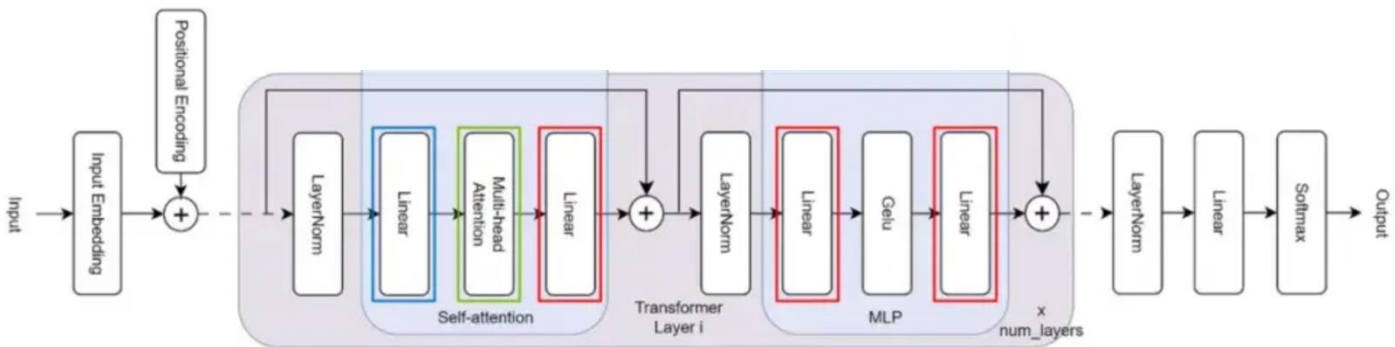
和int8对比, better resolution for non-outlier, per-tensor quantization could keep accuracy

FP8量化:

- 降低耗时
- 降低显存
- 效果接近无损



可量化OPS



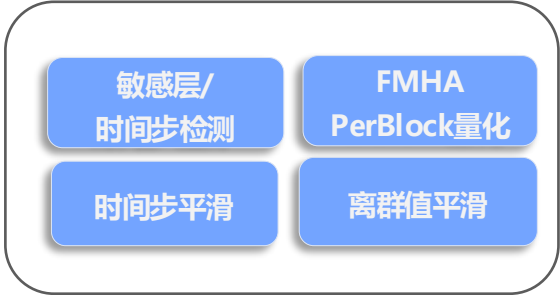
1. QKV GEMM: activation and weight
2. FMHA: scaling factor=1
3. Other GEMM: activation and weight

耗时占比	算子类型	
70%	FMHA	FMHA FP8量化 → 精度损失大, 加速收益高(18%)
11.2%	Nccl 通信	
10%	GEMM	GEMM FP8量化 → 精度损失小, 加速收益小 (5%)

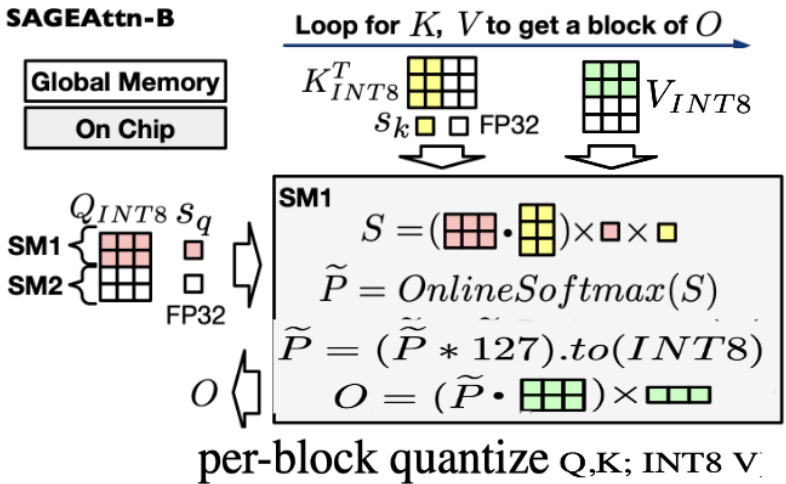
长文Attention计算优化 – FP8计算

Q 量化方式	优点	缺点
Per-Tensor	计算快、存储省	动态范围覆盖差，精度损失大
Per-Token	适配Token间数值差异	存储scale开销与 L 相关
Per-Head	适配Head间注意力分布差异	存储scale开销中等

量化解决方案



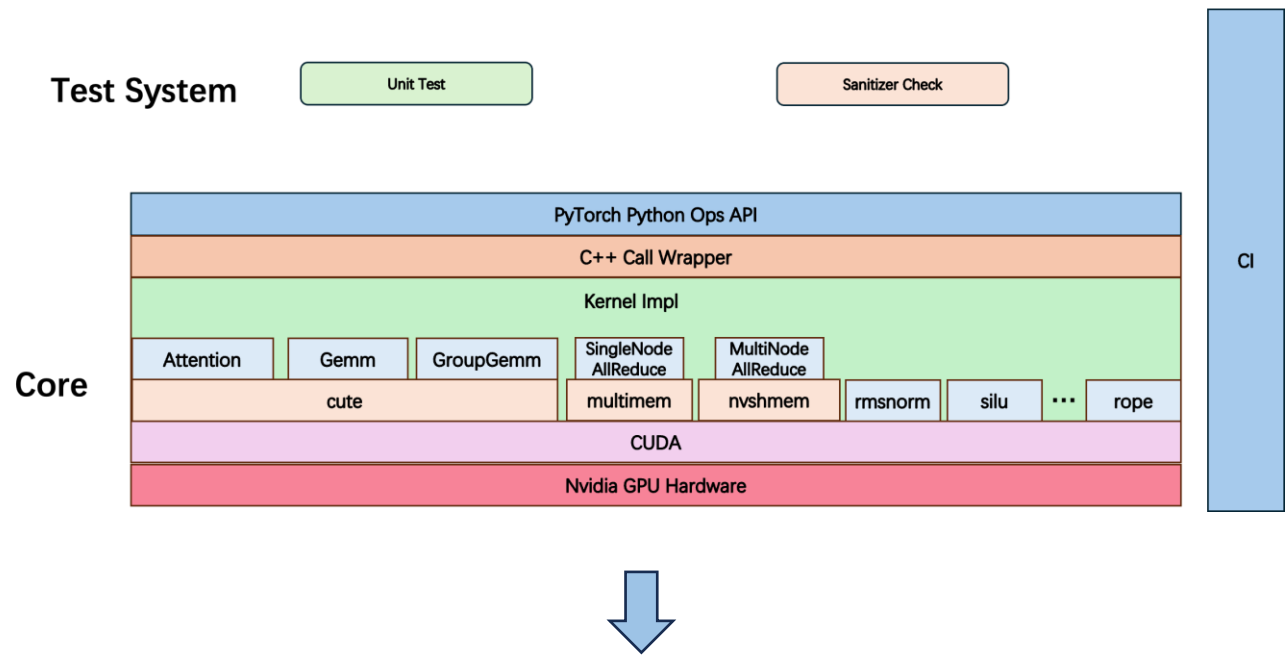
通过 Q 动态量化
Per-Token/Head
高效解决超长SeqLen
量化精度问题



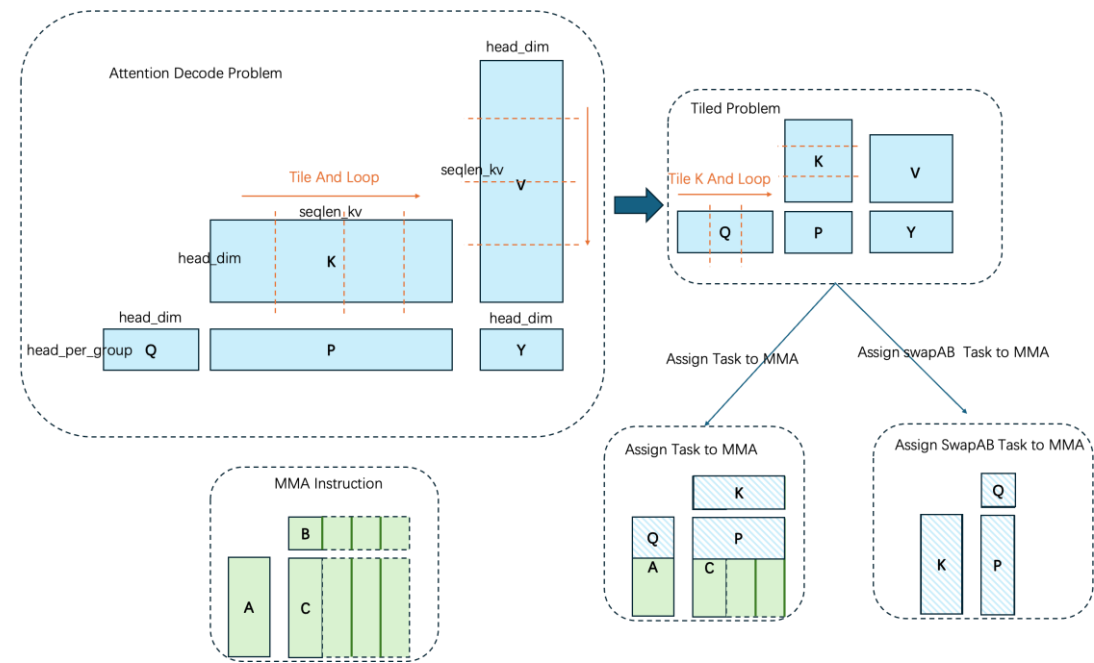
长文Attention计算优化 – HPC kernel

AngelHCF 基于 CUDA 和 CuTe 开发了一套轻量、高效的 LLM 核心算子库
主要包括几大模块：FusedMoE、**Attention**、机内/机间通信、Norm、Sampler、以及各类小算子的融合算子

HPC-Ops 算子库架构图:



项目开源地址: <https://github.com/Tencent/hpc-ops>



Attention SwapAB优化, 访存带宽达 80%+

长文Attention计算优化 – Hopper优化

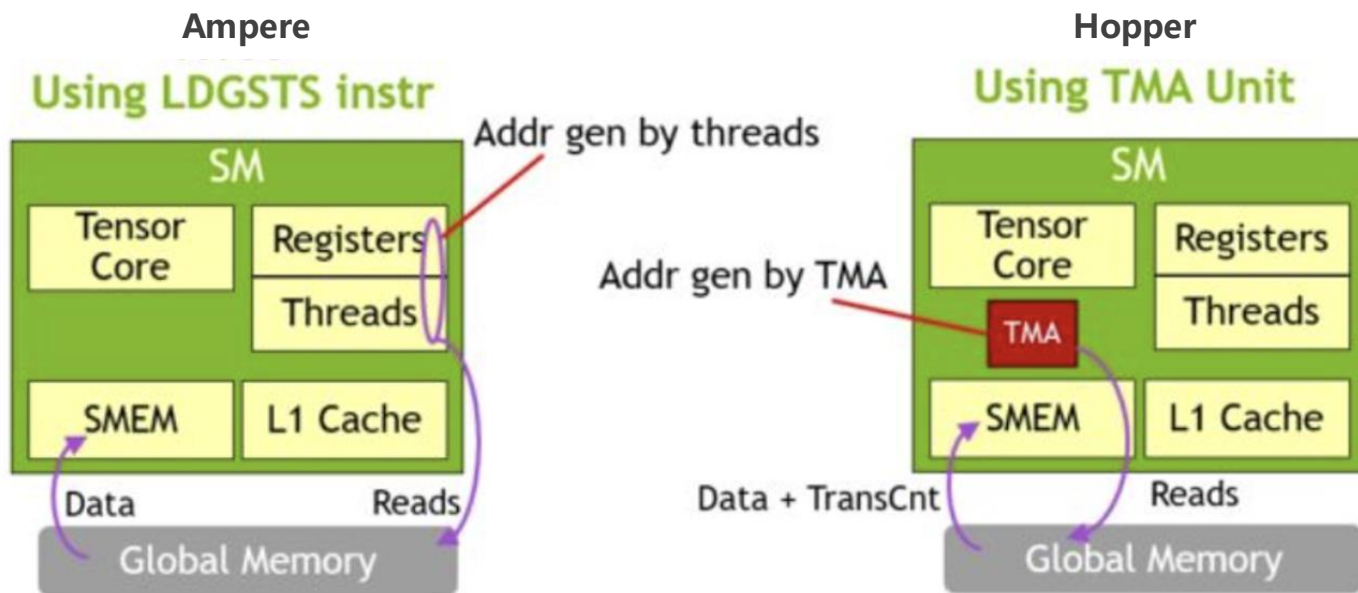
基于Hopper架构和新特性重写Attention Kernel

背景：

- Hopper架构新增TMA允许应用程序在GPU 全局内存和共享内存之间异步且双向地传输一维到五维(1D-5D)的张量、还可以将数据传输到同一线程块集群中其他SM的共享内存(multicast)
- Hopper架构引入WGMMA，支持直接使用共享内存（SMEM）中的数据进行计算，无需经过寄存器文件

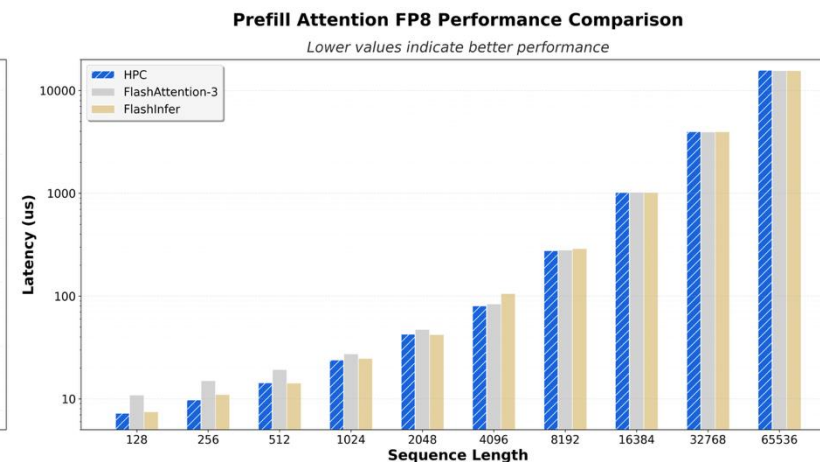
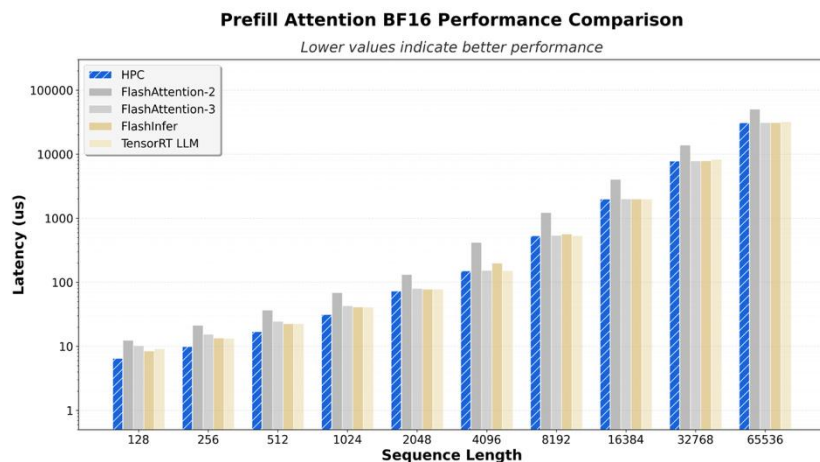
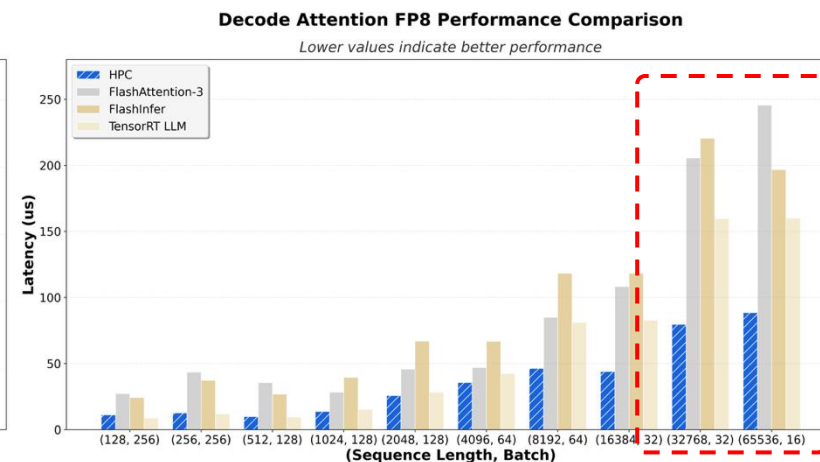
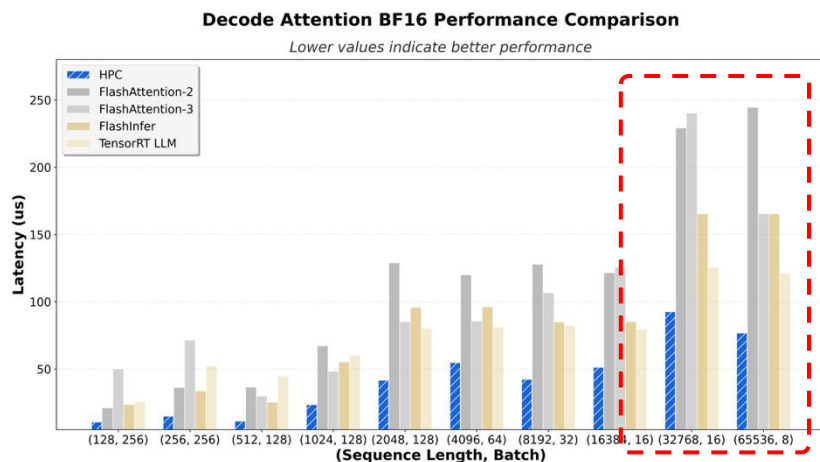
方案：基于Hopper新特性并增加精细化流水线配置和协作线程束组优化，重写MLA、MHA等Attention 算子

收益：MLA 算子性能提升一倍；MHA算子提升20%~80%



长文Attention计算优化 – 性能收益

- 计算耗时上, FP8 prefill attention相比BF16版本, **性能提升87%**;
- FP8 decode attention相比BF16版本性能提升35%, **相比业内 sota版本提升21%**;



大纲目录

腾讯混元及长文场景介绍

稀疏注意力机制探索

长文并行策略优化

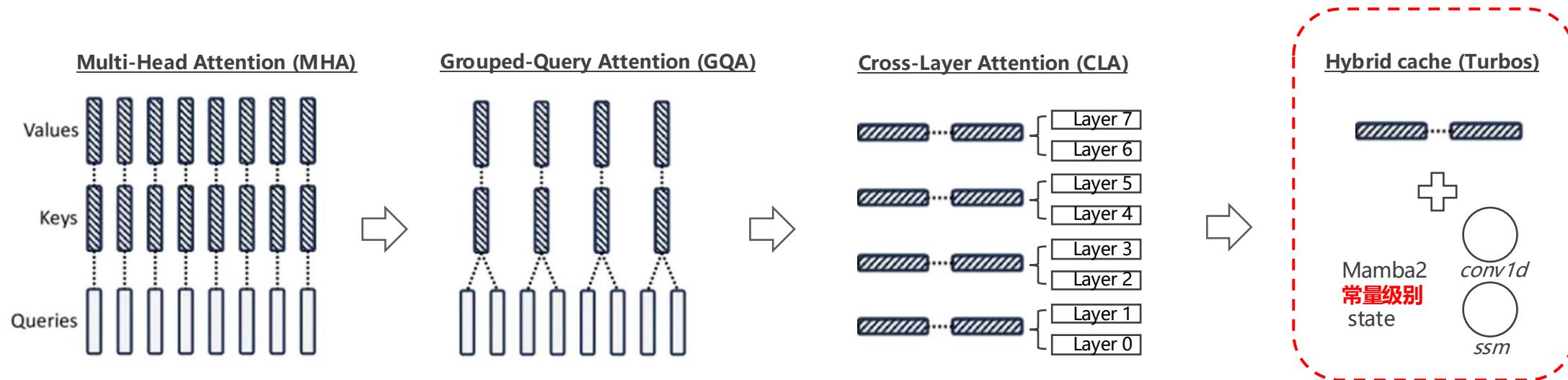
Attention Kernel 计算优化

长文 KVCache 优化

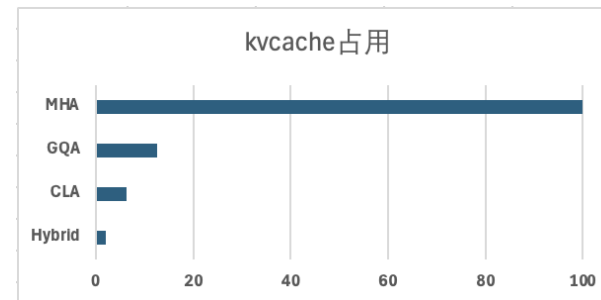
总结

长文KVCache优化 – 模型结构迭代

KVCache 演进, 兼顾效果 + 性能



- 早期混元模型支持 GQA, KVCache 降低为 **1/8~1/10**
- 混元 Large 模型采用 CLA 结构, 再节省**一半** KVCache, Kernel 适配 Prefix cache
- 混元 Turbos 模型 Mamba 层只需要常数级别 state 存储, Cache 整体降低到 **<2%**



长文KVCache优化 – 多级KVCache

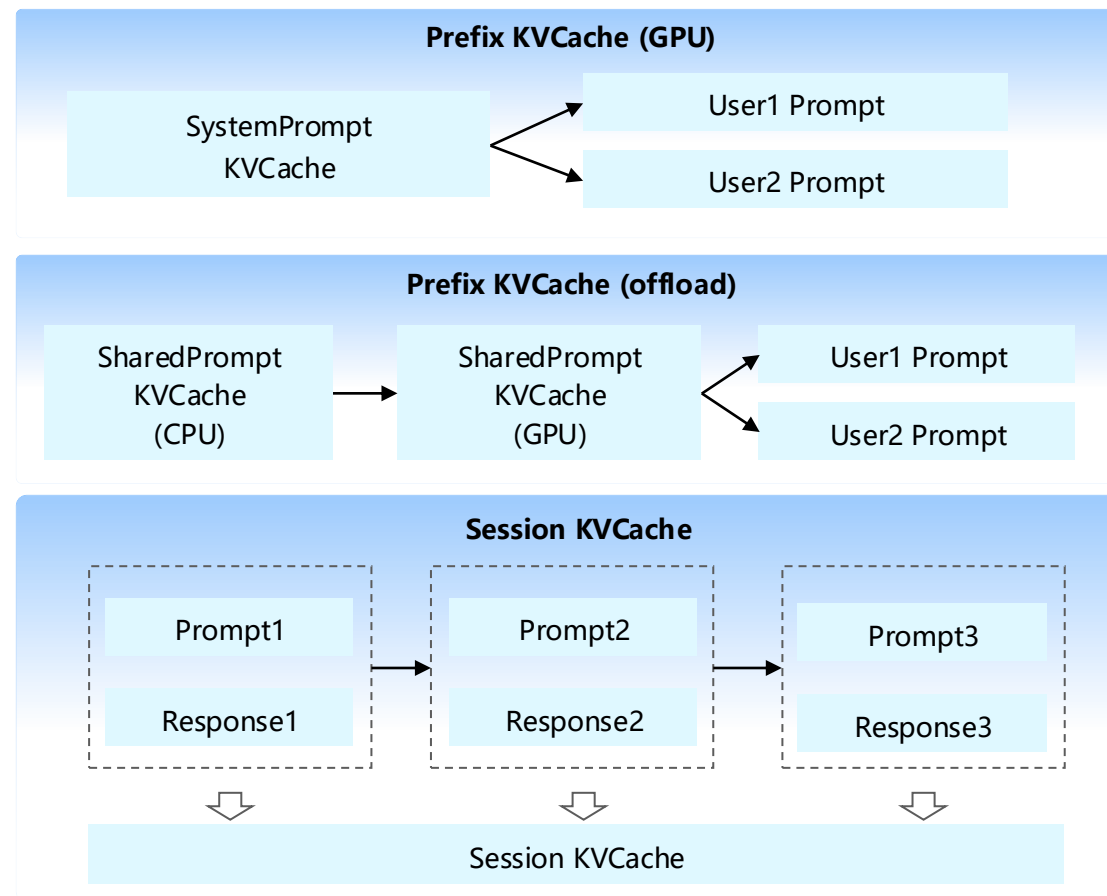
优化 KVCache 调度策略

- 针对输出：
max_performance 调度策略
常见 Paged KVCache 调度：只支持完整调度 or 单步调度
定制以平均输出长度作为调度决策的性能优先调度策略
推理并行 batch 提升 30%+

其他优化 & 探索

- 支持 Mamba Hybrid 结构的 cache reuse
- 探索非 Prefix 前缀匹配的 cache reuse，位置编码部分重新计算后叠加

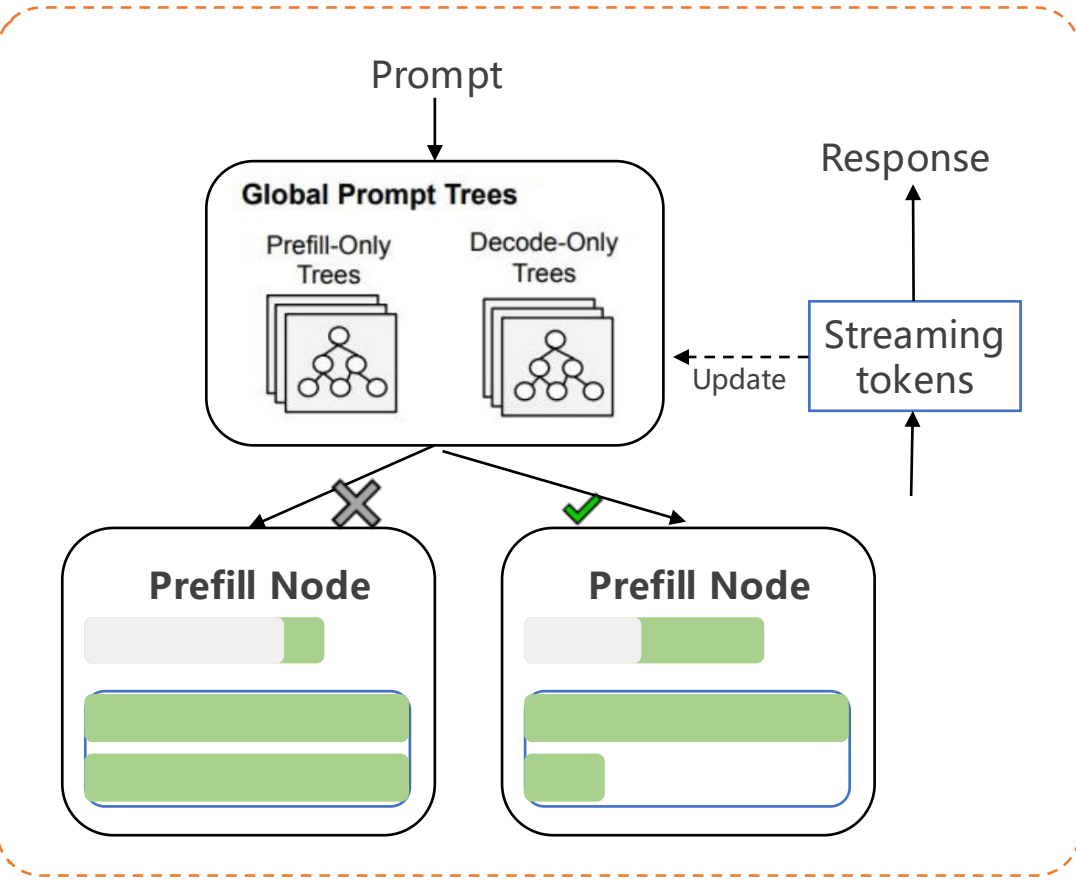
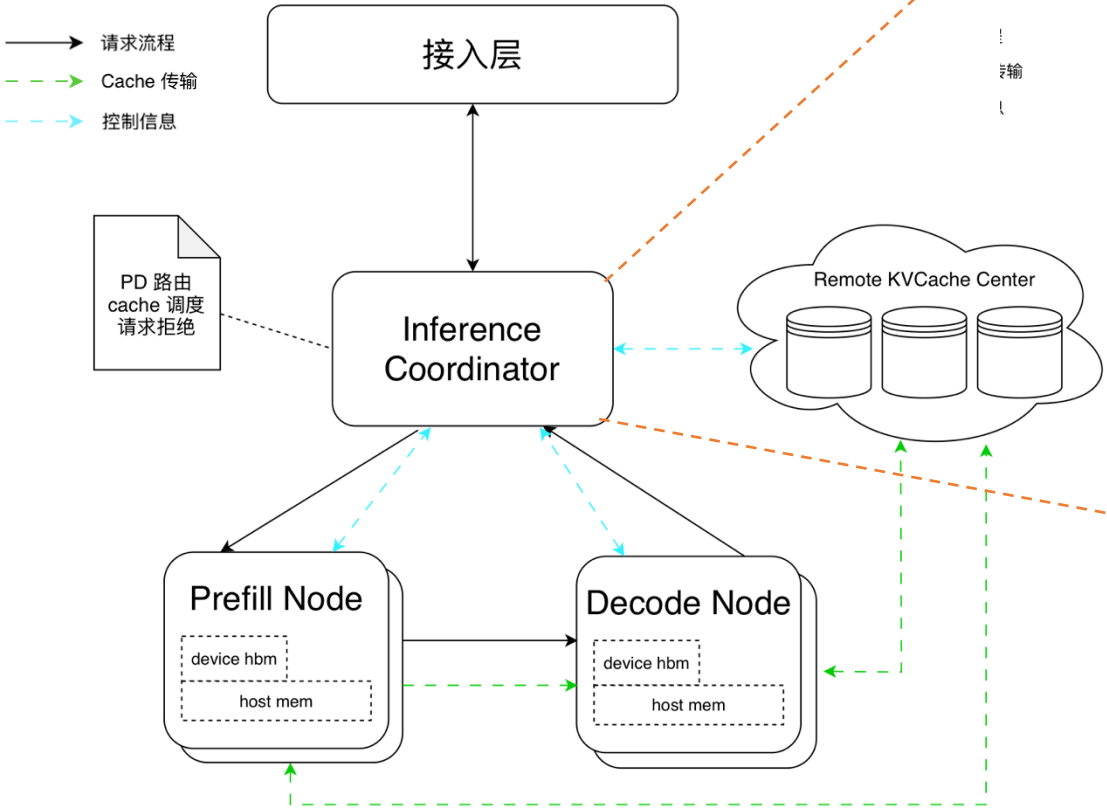
多级 KVCache 缓存机制



长文KVCache优化 – PD调度优化

Prefix Cache 对长文非常关键，需要 Cache aware 智能调度

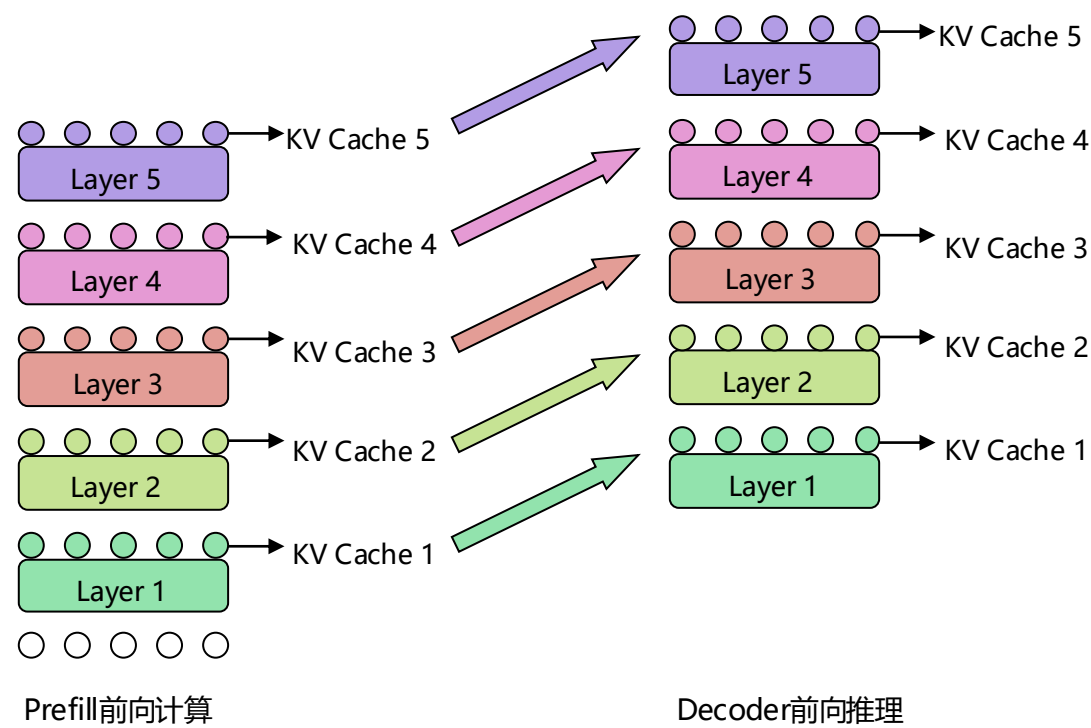
- P 节点结合 Cache reuse、Chunk context 的调度组 Batch 的策略，智能决策调度节点，以及是否远程传输 KVCache，确保更优的 TTFT



收益: 在腾讯某游戏业务上，10k+输入，应用 Prefix cache 以及智能调度策略后，QPM 从优化前的 100 提升到 3000+，几千卡即支撑了峰值 50w QPM 的推广活动

长文KVCache优化 – PD传输优化

- **逐层传输**：相比Prefill完成之后整体发送KV Cache，按层传输能够最小化端到端时延。PDTransfer提供了支持按层传输的接口，框架侧可以一次注册大量内存，根据需要选择每次发送其中的一部分，方便框架侧简洁高效地开发按层传输特性。

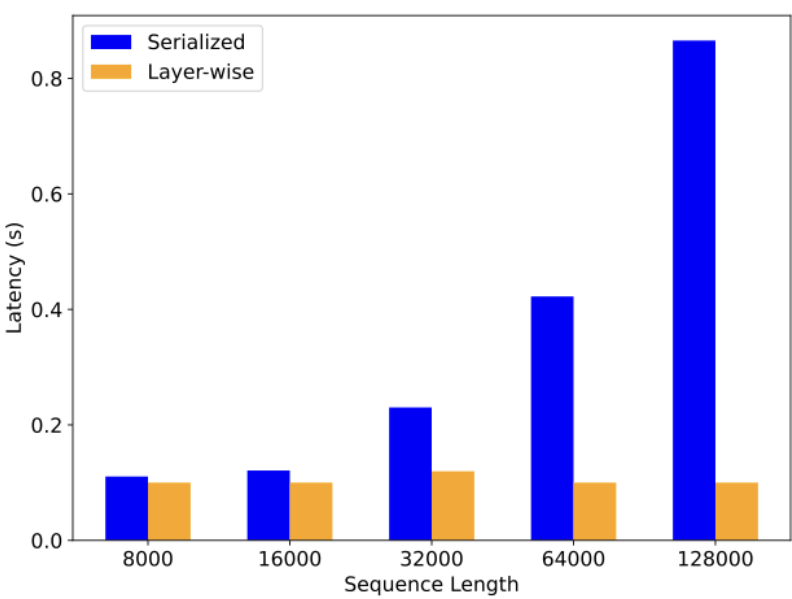


自研PDTransfer，充分利用 RDMA 带宽

➤ *Always Layerwise?*

长文：ISL>8k 时选择 Layerwise 传输

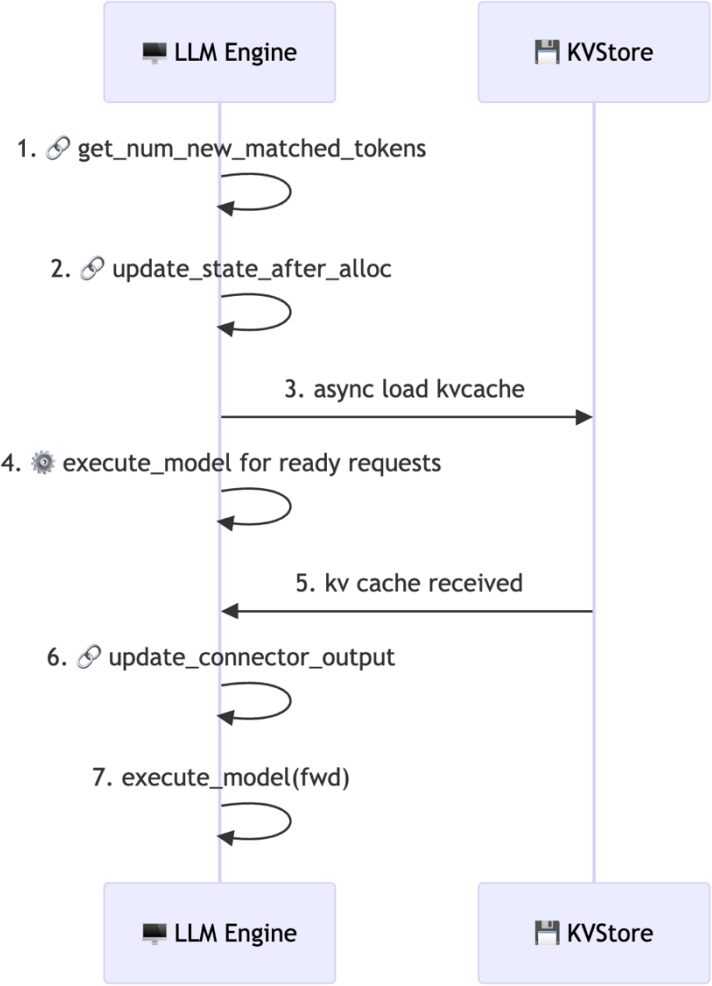
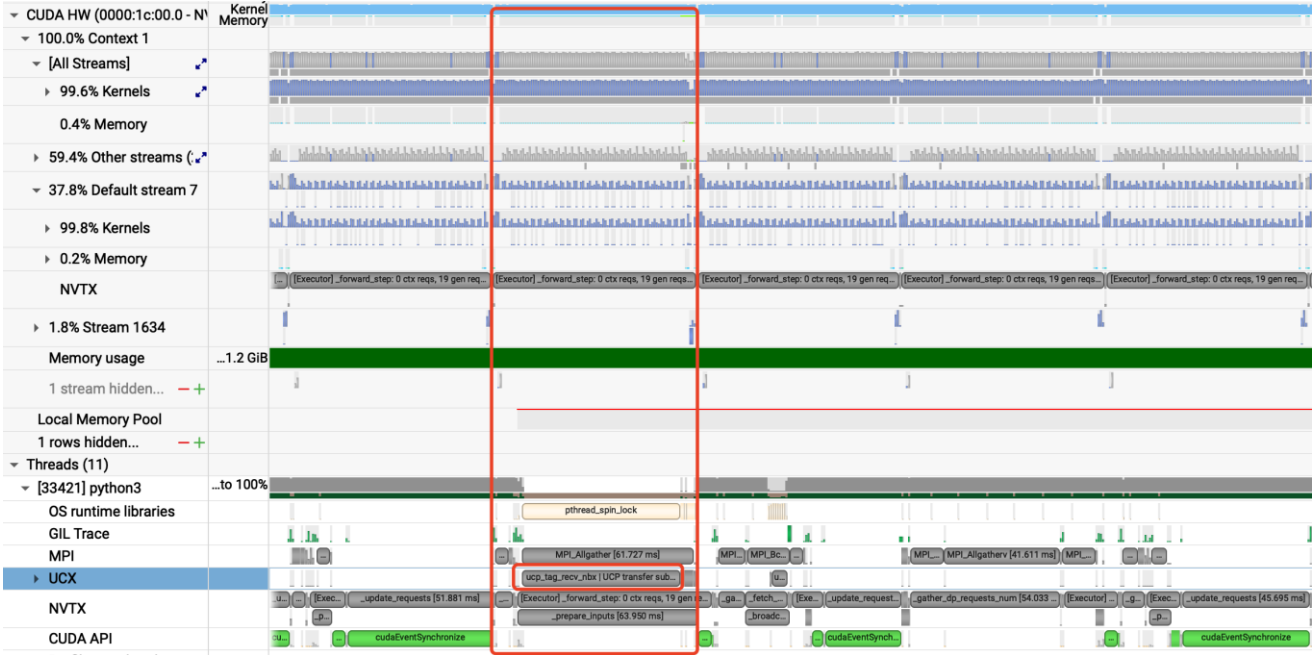
短文：ISL<8k 时整体传输，避免大量小包，并做小包合并传输



长文KVCache优化 – Cache传输优化

异步发送请求级别 Overlap

- 异步传输 KVCache，与当前 Iteration forward step overlap，避免 GPU 空闲等待，提升吞吐



大纲目录

腾讯混元及长文场景介绍

稀疏注意力机制探索

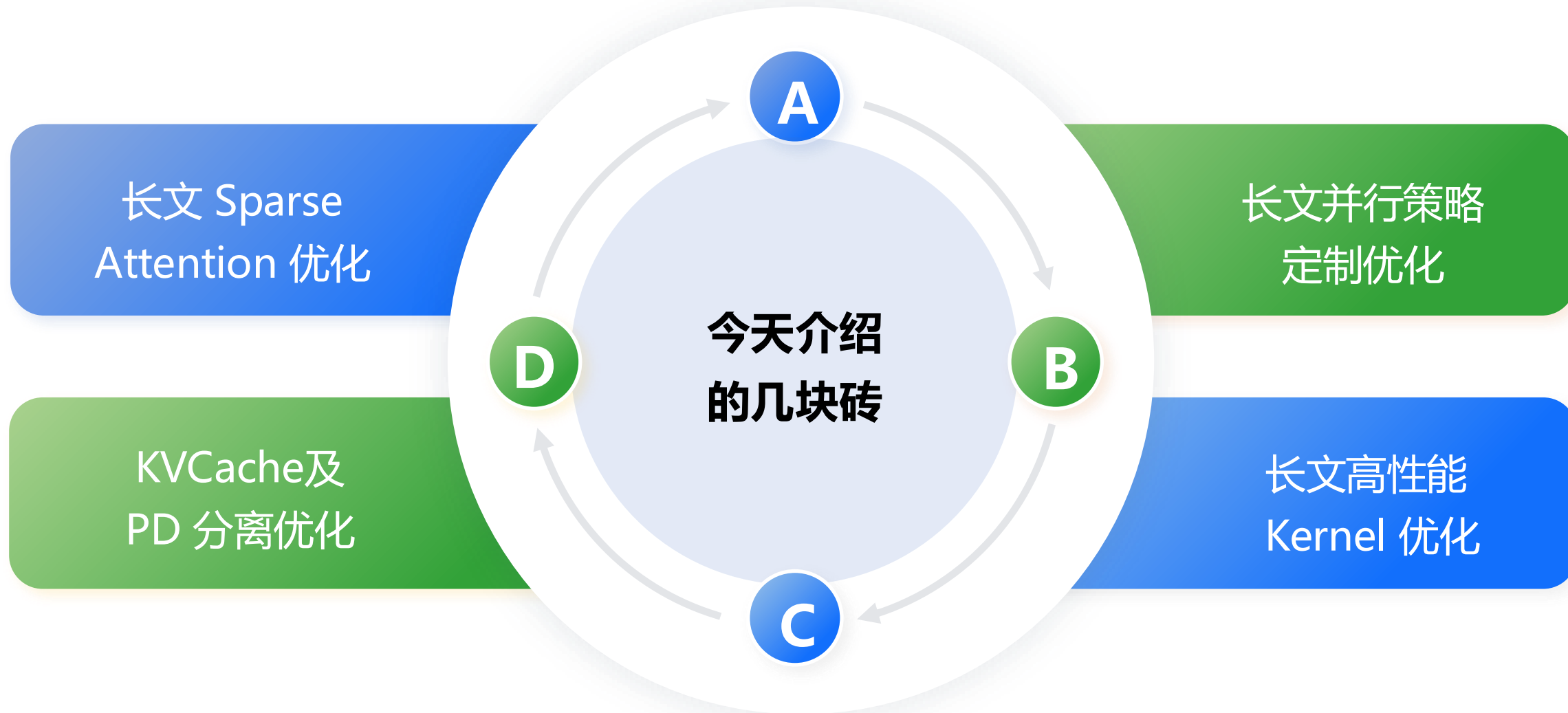
长文并行策略优化

Attention Kernel 计算优化

长文 KVCache 优化

总结

Takeaways



混元大模型已在腾讯大量业务场景落地



腾讯体系内业务广泛落地，助推各产品智能化演进

Thank You !